



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Grafos
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero “B”
Alumnos participantes:	Zamora Arroba Alan Santiago
Asignatura:	Estructura de datos
Docente:	Ing. Caiza Caizabuano Jose Ruben Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Implementar un grafo utilizando lista de adyacencia para representar rutas dentro del Campus Huachi de la UTA y comparar los algoritmos BFS y Dijkstra.

Específicos:

- Completar y documentar los bloques de código (TODO) requeridos en el entorno de desarrollo VS Code para habilitar la inserción de nodos, aristas y las lógicas de exploración de los algoritmos BFS y Dijkstra.
- Comparar analíticamente el comportamiento del algoritmo BFS frente al algoritmo de Dijkstra utilizando un caso de prueba común desde la Universidad hasta el Estadio.
- Estructurar y desplegar el proyecto bajo un sistema de control de versiones en GitHub, asegurando la correcta organización de los archivos fuentes, capturas de pantalla de la consola y la documentación técnica correspondiente.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

- Lleve a cabo las actividades indicadas.
- Suba a la plataforma un informe con el resultado obtenido.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Computador y acceso a internet
- IDE: Visual Studio Code

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____



2.6 Actividades por desarrollar

Implementar el caso de estudio del Mapa del campus y búsqueda de rutas.

2.7 Resultados obtenidos

Se desarrollan habilidades en la solución a un problema de la vida real, en este caso calcular las rutas entre dos ubicaciones, utilizando grafos.

1. Introducción

Los grafos constituyen una estructura de datos fundamental para representar relaciones binarias entre un conjunto de elementos. Un grafo se compone de vértices (o nodos) y aristas (conexiones), las cuales pueden tener pesos asociados para denotar distancias, costos o tiempos de traslado.

Este informe documenta el desarrollo y la culminación de la práctica APE4, enfocada en la simulación de rutas dentro del Campus Huachi de la Universidad Técnica de Ambato (UTA). Para resolver los problemas de conectividad y optimización de trayectorias, se implementó una representación mediante listas de adyacencia y se completaron los algoritmos de búsqueda BFS (Breadth-First Search) y Dijkstra. A través de estos métodos, se contrastan dos enfoques críticos de navegación: la ruta que minimiza la cantidad de escalas o paradas intermedias frente a la ruta que optimiza la distancia geométrica real acumulada.

2. Descripción

El programa modela un grafo no dirigido ponderado que interconecta seis puntos clave del campus universitario: Universidad (uta), FISEI (fisei), Idiomas (idiomas), Biblioteca (biblioteca), Estadio (estadio) y Comedor (comedor).

La arquitectura del código se divide en tres componentes principales y dos lógicas de búsqueda:

Componentes de la Estructura

- **Clase Nodo:** Almacena el identificador único (id) y el nombre legible del lugar (nombre).
- **Clase Arista:** Representa los caminos o enlaces entre locaciones, guardando el nodo destino y el valor numérico del peso (distancia en metros).
- **Clase Grafo:** Gestiona el mapa de nodos y la lista de adyacencia, lo que optimiza el espacio en memoria y facilita el recorrido de los vecinos directos de cualquier punto.



Algoritmos Implementados

1. BFS (Breadth-First Search):

- **Propósito:** Encuentra la ruta con menos paradas (menor número de enlaces), ignorando el peso real de las aristas.
- **Mecanismo:** Utiliza una estructura de tipo Cola (Queue) y un conjunto de Visitados (Set). Explora el grafo nivel por nivel. Al descubrir el nodo destino por primera vez, se garantiza que es el camino con la menor cantidad de saltos posibles.

2. Dijkstra:

- **Propósito:** Encuentra la ruta con la menor distancia total acumulada (mínimo costo).
- **Mecanismo:** Se apoya en una Cola de Prioridad (PriorityQueue) que procesa constantemente el nodo con la distancia más corta conocida desde el origen. Actualiza de manera iterativa los mapas de distancias y nodos anteriores (padres) cuando detecta un camino alternativo más eficiente (relajación de aristas).

Análisis del Caso de Prueba (UTA - Estadio)

El código plantea un escenario diseñado específicamente para evidenciar la diferencia entre ambos criterios:

- **Vía Directa por el Comedor: uta – comedor - estadio.**

Análisis: Tiene solo 2 paradas, pero su distancia total es de $20 + 200 = 220$ unidades.

- **Vía por los Bloques Académicos: uta - fisei - idiomas - biblioteca - estadio.**

Análisis: Tiene 4 paradas, pero sus distancias parciales son menores, sumando un total de $50 + 40 + 30 + 70 = 190$ unidades.

Resultado Esperado de la Ejecución:

- BFS seleccionará la ruta corta en paradas: Universidad - Comedor - Estadio.
- Dijkstra seleccionará la ruta larga en paradas, pero corta en metros: Universidad - FISEI - Idiomas - Biblioteca - Estadio.



3. Explicación del código

```
8      static class Nodo {
9          String id;
10         String nombre;
11
12         public Nodo(String id, String nombre) {
13             this.id = id;
14             this.nombre = nombre;
15         }
16     }
```

Representa un lugar físico dentro de la universidad (un vértice del grafo).

- **id:** Es una clave corta y única para buscarlo rápido.
- **nombre:** Es el texto amigable para mostrar al usuario.

```
21     static class Arista {
22         String destino;
23         int peso;
24
25         public Arista(String destino, int peso) {
26             this.destino = destino;
27             this.peso = peso;
28         }
29     }
```

Representa una calle o sendero que conecta a un nodo con otro.

destino: El id del nodo al que se llega a través de este camino.

peso: El costo de transitarlo. En este caso, representa la distancia en metros entre los dos puntos.

```
34     static class Grafo {
35
36         Map<String, Nodo> nodos = new HashMap<>();
37         Map<String, List<Arista>> adyacencia = new HashMap<>();
```

- **Nodos:** Un diccionario (HashMap) que asocia el id (String) con el objeto Nodo completo. Facilita obtener el nombre de un lugar sabiendo solo su inicial o código.
- **Adyacencia:** Es la Lista de Adyacencia. Para cada id de un nodo, guarda una lista de todos los vecinos que tiene conectados directamente.



```
43     public void agregarNodo(String id, String nombre) {
44         // Se registra el nodo en el mapa principal usando su id como clave
45         nodos.put(id, new Nodo(id, nombre));
46         // Se inicializa su lista de vecinos vacía en el mapa de adyacencia
47         adyacencia.put(id, new ArrayList<>());
48     }
49
50     // =====
51     // TODO 2
52     // Agregar arista no dirigida
53     // =====
54     public void agregarArista(String origen, String destino, int peso) {
55         // Como el grafo es no dirigido, la conexión se añade en ambos sentidos
56         adyacencia.get(origen).add(new Arista(destino, peso));
57         adyacencia.get(destino).add(new Arista(origen, peso));
58     }
```

- **AgregarNodo:** Guarda el lugar en el mapa de nodos y le prepara una lista vacía en el mapa de adyacencia para meter allí a sus futuros vecinos.
- **AgregarArista:** Como en el campus puedes caminar de ida y de vuelta por el mismo pasillo, el código añade la conexión en ambos sentidos: de origen a destino y de destino a origen.



```
64 public List<String> bfs(String inicio, String fin) {
65
66     // Cola para recorrer niveles
67     Queue<List<String>> cola = new LinkedList<>();
68
69     // Nodos visitados
70     Set<String> visitados = new HashSet<>();
71
72     // Camino inicial
73     List<String> caminoInicial = new ArrayList<>();
74
75     // TODO: Agregar nodo inicio al camino inicial
76     caminoInicial.add(inicio);
77
78     // TODO: Agregar caminoInicial a la cola
79     cola.add(caminoInicial);
80
81     // TODO: Marcar inicio como visitado
82     visitados.add(inicio);
83
84     while (!cola.isEmpty()) {
85
86         // TODO: Obtener el primer camino de la cola
87         List<String> camino = cola.poll();
88
89         // Nodo actual
90         String actual =
91             camino.get(camino.size() - 1);
92
93         // Si llegamos al destino
94         if (actual.equals(fin)) {
95             return camino;
96         }
97
98         // Recorrer vecinos
99         for (Arista arista : adyacencia.get(actual)) {
100
101             // TODO: Verificar si el vecino NO fue visitado
102             if (!visitados.contains(arista.destino)) {
103
104                 // TODO: Marcar vecino como visitado
105                 visitados.add(arista.destino);
106
107                 // Crear nuevo camino
108                 List<String> nuevoCamino =
109                     new ArrayList<>(camino);
110
111                 // TODO: Agregar vecino al nuevo camino
112                 nuevoCamino.add(arista.destino);
113
114                 // TODO: Agregar nuevoCamino a la cola
115                 cola.add(nuevoCamino);
116             }
117         }
118     }
119
120     return null;
121 }
```

Guarda caminos enteros dentro de la Queue. Como procesa primero los caminos de 1 salto, luego los de 2, luego los de 3, el primero en llegar a la meta es, por obligación, el que menos paradas hace.



```
127 public List<String> dijkstra(String inicio, String fin) {
128
129     Map<String, Integer> distancias =
130         new HashMap<>();
131
132     Map<String, String> anteriores =
133         new HashMap<>();
134
135     PriorityQueue<String> cola =
136         new PriorityQueue<>(  
137             Comparator.comparingInt(  
138                 distancias::get  
139             )  
140         );
141
142     // Inicializar distancias
143     for (String nodo : nodos.keySet()) {
144         // TODO: Inicializar distancia infinita
145         distancias.put(nodo, Integer.MAX_VALUE);
146     }
147
148     // TODO: Distancia del inicio = 0
149     distancias.put(inicio, value: 0);
150
151     // TODO: Agregar inicio a la cola
152     cola.add(inicio);
153
154     while (!cola.isEmpty()) {
155
156         // TODO: Obtener nodo con menor distancia
157         String actual = cola.poll();
158
159         // Optimización: si alcanzamos el nodo fin, podemos terminar el bucle
160         if (actual.equals(fin)) break;
161
162         for (Arista arista : adyacencia.get(actual)) {
163
164             // TODO: Calcular nueva distancia
165             int nuevaDistancia = distancias.get(actual) + arista.peso;
166
167             // TODO: Verificar si nuevaDistancia es menor
168             if (nuevaDistancia < distancias.get(arista.destino)) {
169
170                 // TODO: Actualizar distancia
171                 distancias.put(arista.destino, nuevaDistancia);
172
173                 // TODO: Guardar nodo anterior
174                 anteriores.put(arista.destino, actual);
175
176                 // TODO: Agregar vecino a la cola
177                 cola.add(arista.destino);
178             }
179         }
180     }
181
182     // Reconstruir camino
183     List<String> camino = new ArrayList<>();
184
185     String actual = fin;
186
187     while (actual != null) {
188
189         camino.add(index: 0, actual);
190
191         actual = anteriores.get(actual);
192     }
193
194     return camino;
```

La Relajación de aristas. Si para ir a la biblioteca descubres que es más rápido ir dando un rodeo por tres bloques académicos que ir por el camino directo de tierra, Dijkstra actualiza ese valor inmediatamente. Al final reconstruye la ruta yendo del fin hacia el inicio usando el mapa de anteriores.



```
229 public static void main(String[] args) {
230
231     Grafo grafo = new Grafo();
232
233     // NODOS
234     grafo.agregarNodo(id: "uta", nombre: "Universidad");
235     grafo.agregarNodo(id: "fisei", nombre: "FISEI");
236     grafo.agregarNodo(id: "idiomas", nombre: "Idiomas");
237     grafo.agregarNodo(id: "biblioteca", nombre: "Biblioteca");
238     grafo.agregarNodo(id: "estadio", nombre: "Estadio");
239     grafo.agregarNodo(id: "comedor", nombre: "Comedor");
240
241     // ARISTAS
242     grafo.agregarArista(origen: "uta", destino: "fisei", peso: 50);
243     grafo.agregarArista(origen: "fisei", destino: "idiomas", peso: 40);
244     grafo.agregarArista(origen: "idiomas", destino: "biblioteca", peso: 30);
245     grafo.agregarArista(origen: "biblioteca", destino: "estadio", peso: 70);
246
247     // Ruta con menos paradas
248     // pero más distancia
249     grafo.agregarArista(origen: "uta", destino: "comedor", peso: 20);
250     grafo.agregarArista(origen: "comedor", destino: "estadio", peso: 200);
251
252     // =====
253     // PRUEBAS
254     // =====
255
256     System.out.println(x: "==== BFS =====");
257
258     List<String> rutaBFS =
259     |     grafo.bfs(inicio: "uta", fin: "estadio");
260
261     grafo.mostrarRuta(rutaBFS);
262
263     System.out.println(x: "\n==== DIJKSTRA =====");
264
265     List<String> rutaDijkstra =
266     |     grafo.dijkstra(inicio: "uta", fin: "estadio");
267
268     grafo.mostrarRuta(rutaDijkstra);
269 }
270 }
```

- **BFS:** ve la ruta uta - comedor - estadio, cuenta 2 nodos y dice: "Esta es la mejor". No le importa que caminar por ahí sume un cansancio de 220 metros.
- **Dijkstra:** calcula los metros de ambas opciones. Compara 220 metros (por el comedor) contra 190 metros (por los bloques). Elige la de 190 metros, a pesar de que requiera caminar por 4 paradas distintas.

4. Herramientas utilizadas

- Laptop
- JDK
- Visual Studio Code
- GitHub



5. Resultados

```
PS C:\Users\ASUS\Documents\PROYECTO APE4> cd "c:\Users\ASUS\Documents\PROYECTO APE4\src\" ; if ($?) { javac APE4_Grafos.java } ; if ($?) {  
    } { java APE4_Grafos }  
==== BFS ====  
Universidad (uta) -> Comedor (comedor) -> Estadio (estadio)  
  
==== DIJKSTRA ====  
Universidad (uta) -> FISEI (fisei) -> Idiomas (idiomas) -> Biblioteca (biblioteca) -> Estadio (estadio)  
PS C:\Users\ASUS\Documents\PROYECTO APE4\src>
```

- **BFS** imprime la ruta a través del Comedor (Universidad - Comedor - Estadio) porque su único criterio de optimización es minimizar el número de conexiones o escalas entre el origen y el destino, ignorando por completo las distancias reales. Bajo esta lógica "ciega" a los kilómetros, el algoritmo prefiere un trayecto de solo dos tramos, aunque implique caminar una distancia total más larga de 220 metros, ya que para sus ojos es la vía más directa en cuanto a paradas.
- **Dijkstra** imprime la ruta larga en paradas, pero eficiente en distancia (Universidad - FISEI - Idiomas - Biblioteca - Estadio) debido a que analiza y suma minuciosamente los metros individuales de cada sendero para encontrar el costo total más bajo. Al contrastar el mapa, Dijkstra descubre matemáticamente que dar el rodeo por los bloques académicos e idiomas suma un total de 190 metros, demostrando que es una ruta mucho más corta y ahorrativa (30 metros menos) que la vía del comedor, a pesar de requerir más estaciones intermedias.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Se completaron con éxito los bloques TODO en VS Code, logrando que el grafo gestione correctamente los nodos y caminos mediante listas de adyacencia y el uso eficiente de colecciones en Java.
- La comparación demostró que BFS solo reduce el número de paradas, mientras que Dijkstra prioriza el ahorro de distancia, hallando una ruta más larga en paradas pero más corta en recorrido.
- El proyecto se respaldó y organizó correctamente en GitHub bajo la estructura exigida, garantizando un correcto control de versiones y la entrega transparente de las evidencias.



2.10 Recomendaciones

- Añadir validaciones en el código para verificar que los nodos existan antes de unirlos, evitando que el programa falle si se introduce un lugar por error.
- Aplicar BFS si en el futuro se busca diseñar rutas de buses internos y Dijkstra para guiar a peatones que buscan caminar lo menos posible.

2.11 Referencias bibliográficas

Repositorio de GitHub:

<https://github.com/AlanZamora10/Proyecto-APE4.git>

2.12 Anexos

```
src > J APE4_Grafos.java
1  import java.util.*;
2
3  public class APE4_Grafos {
4
5      //
6      // Nodo
7      //
8      static class Nodo {
9          String id;
10         String nombre;
11
12         public Nodo(String id, String nombre) {
13             this.id = id;
14             this.nombre = nombre;
15         }
16     }
17
18     //
19     // Arista
20     //
21     static class Arista {
22         String destino;
23         int peso;
24
25         public Arista(String destino, int peso) {
26             this.destino = destino;
27             this.peso = peso;
28         }
29     }
30
31     //
32     // Grafo
33     //
34     static class Grafo {
35
36         Map<String, Nodo> nodos = new HashMap<>();
37         Map<String, List<Arista>> adyacencia = new HashMap<>();
38
39         //
40         // TODO 1
41         // Agregar nodo al grafo
42         //
43         public void agregarNodo(String id, String nombre) {
44             // Se registra el nodo en el mapa principal usando su id como clave
45             nodos.put(id, new Nodo(id, nombre));
46             // Se inicializa su lista de vecinos vacía en el mapa de adyacencia
47             adyacencia.put(id, new ArrayList<>());
48         }
49     }
50 }
```

Proyecto-APE4 Public

Pin Watch 0

main

1 Branch

0 Tags

Go to file

T

Add file

Code

AlanZamora10

APE completa

6107fff · 8 hours ago

1 Commit

captura

APE completa

8 hours ago

src

APE completa

8 hours ago

README.md

APE completa

8 hours ago